

Dart: 基本語法(1)

建立第一支 Dart 應用程式

- Task1: 首先確認 Dart SDK 是否已安裝妥當
 - 打開 VSCode -> Terminal -> New Terminal
 - 輸入下列指令，確認 Dart SDK 版本：

```
dart --version
```

- 出現如下的訊息，表示已安裝 Dart SDK

```
Dart SDK version: 3.9.2 (stable) (Wed Aug 27 03:49:40 2025 -0700) on "win
```

- Task2: 建立一個新的 Dart 專案
 - 在 Terminal 視窗內，建立一個新的資料夾 dartex1，並且切換到該目錄下：

```
mkdir dartex1  
cd dartex1
```

- 執行下列指令：

```
dart create cli
```

- Dart 將在 cli 目錄底下，進行目錄配置與相關檔案的建立與初始化
- 初始化結束後，會在 cli 目錄底下，看到下列訊息內容：

```
Creating cli using template console...
```

```
.gitignore  
analysis_options.yaml  
CHANGELOG.md  
pubspec.yaml  
README.md  
bin\cli.dart  
lib\cli.dart  
test\cli_test.dart
```

```
Running pub get...
```

```
Resolving dependencies...  
Downloading packages...  
Changed 48 dependencies!
```

```
Created project cli in cli! In order to get started, run the following
```

[Back to top](#)

```
cd cli
dart run
```

- Task3: 執行第一個 Dart 程式
 - 切換到 cli 目錄下：

```
cd cli
```

- 執行預設的應用程式

```
dart run
```

- 看到下列訊息：

```
Building package executable...
Built cli:cli.
Hello world: 42!
```

- Task4: 修改程式內容
 - 利用 VSCode ，打開 bin/cli.dart 檔案
 - 確認檔案內容如下所列：

```
import 'package:cli/cli.dart' as cli;

void main(List<String> arguments) {
  print('Hello world: ${cli.calculate()}!');
}
```

- 程式進入點在 main 函式內
- 將程式改寫成如下內容：

```
// import 'package:cli/cli.dart' as cli; // 註解本行

void main(List<String> arguments) {
  print('Hello, Dart!'); // 改變本行內容
}
```

- 存檔後，再次執行程式：

```
dart run
```

- 執行結果應如下所示：

```
Building package executable...
Built cli:cli.
```

```
Hello, Dart!
```

在應用程式中加入互動

- Task1: 執行 version 以及 help 指令參數
 - 在 bin/cli.dart 檔案中，修改檔案內容：

```
const version = '0.0.1';

void main(List<String> arguments) {
  if (arguments.isEmpty) {
    print('Hello, Dart!');
  } else if (arguments.first == 'version') {
    print('Dartpedia CLI Version: $version');
  }
}
```

- const 語法用在定義不可變動的常數內容。
- version 為一變數名稱，呼叫時，需使用「\$」字符。
- 每行程式語法結束，均使用「;」字符做結尾。
- Dart 的條件判斷語法：

```
// 只有 if 敘述句
if (條件判斷式) {
  成功時執行工作;
}

// 加上 else 進行失敗時的執行內容
if (條件判斷式) {
  成功時執行工作;
} else {
  失敗時執行工作;
}

// 進行二次再判斷的方式
if (條件判斷式) {
  成功時執行工作;
} else if (條件判斷式) {
  成功時執行工作
} else {
  失敗時執行工作;
}
```

- 存檔後，執行該程式

```
dart bin/cli.dart version
```

- 將出現如下結果：

[Back to top](#)

Dartpedia CLI Version: 0.0.1

- 在 bin/cli.dart 檔案內，於 main() 函式下方，加入下列程式內容：

```
(前方省略...)
void printUsage() { // Add this new function
  print(
    "The following commands are valid: 'help', 'version', 'search <ARTICLE-TITLE>'");
}
```

- 在 main() 函式內，將程式修改成下列內容：

```
(前方省略...)
void main(List<String> arguments) {

  if (arguments.isEmpty || arguments.first == 'help') {

    printUsage(); // 呼叫 printUsage() 函式程式

  } else if (arguments.first == 'version') {

    print('Dartpedia CLI version $version');

  } else {

    printUsage(); // 若有未指定的參數被輸入，呼叫 printUsage() 函式處理

  }
}
(後方省略...)
```

- arguments：被指定為輸入資料的變數名稱，其資料型態為 List 數值。
- arguments.isEmpty：確認該輸入的資料是否有值
- version 變數名稱被宣告成 const 資料型態，其內容值不得改變

- 存檔後，執行該程式

```
dart bin/cli.dart help
```

- 將出現如下結果：

```
The following commands are valid: 'help', 'version', 'search <ARTICLE-TITLE>'
```

- Task2: 執行 search 指令參數，取得文章的標頭
 - 將 search 字眼加入 main() 主要函式內，修改 bin/cli.dart 檔案內容成如下所示：

```
(前方省略...)
void main(List<String> arguments) {
  if (arguments.isEmpty || arguments.first == 'help') {
    printUsage();
  } else if (arguments.first == 'version') {
    print('Dartpedia CLI version $version');

    // 新增下列兩行
  } else if (arguments.first == 'search') {
    print('Search command recognized!');

  } else {
    printUsage();
  }
}
(後方省略...)
```

- 執行下列指令，測試 search 是否可行

```
dart bin/cli.dart search
```

- 應該會看到如下訊息：

```
Search command recognized!
```

- 定義 searchWikipedia 函式，在接受參數之後，可列出要求查詢的關鍵字眼：

```
(前方省略...)

// 在 main() 函式下方，新增下列函式內容，引數的宣告上，使用 ? 表示輸入的資料型態
void searchWikipedia(List<String>? arguments) {
  print('searchWikipedia received arguments: $arguments');
}

(後方接 printUsage() 函式，故省略...)
```

- List? arguments：表示參數列可以是任意內容，也可以空值。
 - 但未宣告的變數值，是不能輸入到該參數列中。
- 修改 main() 函式內容，加入呼叫 searchWikipedia 至 search 判斷式程式內容：

```
(前方省略...)

// 修改 search 判斷式程式內容
} else if (arguments.first == 'search') {

  // 加入三元判斷式，在判斷後，呼叫 searchWikipedia 函式
  final inputArgs = arguments.length > 1 ? arguments.sublist(1) : null;
  searchWikipedia(inputArgs);
```

[Back to top](#)

```
} else {
```

(後方省略...)

- 三元判斷式語法：

```
判斷式條件 ? 符合 : 不符合;
```

- `final`: 指定變數內容只能一次性輸入，之後不得再修改。
- `arguments.sublist(1)`: 建立一個新的資料列，包含第一個元素之後的指定元素行內所有的元素。
- 執行下列指令，測試 `search` 是否可行

```
dart bin/cli.dart search Dart Programming
```

- 應該會看到如下訊息：

```
searchWikipedia received arguments: [Dart, Programming]
```

- 此時，`$arguments` 參數所取得的資料，應該是一維的陣列資料：`[Dart, Programming]`
- 再執行下列指令，測試 `search`

```
dart bin/cli.dart search
```

- 應該會看到如下訊息：

```
searchWikipedia received arguments: null
```

- 進一步修改 `main()` 函式內容，讓程式可利用標準輸入(`stdin`)方式，接受輸入的文件：

```
// 修改 bin/cli.dart 檔案，在第一行加入下列程式碼
import 'dart:io';

(中間省略...)

// 修改 searchWikipedia 函式內容
void searchWikipedia(List<String>? arguments) {
  final String articleTitle;

  // 判斷是否沒有輸入任何文字
  if (arguments == null || arguments.isEmpty) {
    print('Please provide an article title.');
```

```
    // 假如沒有輸入任何文字，就等待使用者需要輸入文章的標頭
    articleTitle = stdin.readLineSync() ?? '';

  } else {
```

[Back to top](#)

```
// 否則，就自行加入一個空白字元
articleTitle = arguments.join(' ');
}

// 此處加入提示語句
print('Looking up articles about "$articleTitle". Please wait.');
```

```
print('Here you go!');
```

```
print('Current article title: $articleTitle');
```

```
}
```

(後方省略...)

- [dart.io](https://dart.dev) 是 Dart SDK 的核心函式，提供處理檔案、目錄、socket、HTTP 用戶端與伺服器端的 API 程式。
- `final String articleTitle`: 定義後方的變數是只能一次性輸入的文字字串。
- `stdin.readLineSync() ?? ''`: 讀取使用者輸入的字串。
 - 但如果沒輸入，則利用非合併操作子「??」提供一個空白的字串，確保回傳值在 null 情況下，一定有文字內容。
 - `arguments.join(' ')`: 加入一個空白字元到輸入的資料內，當作是分隔符號。
- 執行下列指令進行測試

```
dart bin/cli.dart search Dart Programming
```

- 應該會看到如下訊息：

```
Looking up articles about "Dart Programming". Please wait.
Here you go!
Current article title: Dart Programming
```

參考文獻

- [Dart 官方教學手冊](#)

Generated by [aglio](#) on 09 Apr 2026